

The match functionality

```
Route::match(['get', 'post'], '/', function () {
    // });
```

The any functionality

```
Route::any('foo', function () {
    // });
```

Now let's see route codes in action. The first program has to be 'Hello World'. The syntax would be

```
Route::get('foo', function () {
    return 'Hello World'; });
```

This simple first code that you run when start learning Laravel (as is the norm with any programming language you learn) returns the output of (you guessed it!) 'Hello World' on your screen.

The code

```
Route::get('/', function () {
    return view('welcome'); });
```

This code will display whatever has been defined in the welcome.blade.php file. Breaking it down, the code told the system that the method is get and the root URL is '/'. This executes the related function. Next, it pointed to the file welcome.blade.php (which is entered without the .blade.php suffix) and to view that file which completed the desired action.

You can also use the view method to directly show or return a view or a display by using the `Route::view` command.

```
E.g. Route::view('/welcome', 'welcome');
```

This can be additionally modified to include one more argument in the function like

```
Route::view('/welcome', 'welcome', ['name' => 'Digit']);
```

This is a handy shortcut that reduces the lines of code you need to enter.

Middleware

Think of middleware as a security guard who opens the door only for authorised personnel. If you are not authorised, you will be sent to the HR to get authorisation. In the way the security guard acts as a filter in the physical world, middleware acts as a filter in the web application world. It filters all Http requests entering the web application. Depending on the the conditions specified, it can either allow the request to go through or cancel it or redirect it. For e.g. if a guest tries to post a blog without logging